



UTM^{online}

TECNOLOGÍAS DE LA INFORMACIÓN EN LINEA

SISTEMAS EMBEBIDOS

3 créditos



Profesor :

Ing. Freddy Carrera Sanchez, Msc

Titulaciones	Semestre
Ingeniero en Tecnologías de la Información	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje (online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección **CAFETERÍA VIRTUAL**.

PERÍODO ABRIL – AGOSTO 2026

Índice

Tabla de contenido

Unidad 1. FUNDAMENTOS DE SISTEMAS EMBEBIDOS	3
Tema 1.1: Conceptos Básicos de Sistemas Embebidos	3
1.1.1 Características esenciales de los sistemas embebidos.....	3
1.1.2 Breve historia de los sistemas embebidos	6
1.1.3 Tipos de Sistemas Embebidos	8
1.1.4 Aplicación de Sistemas Embebidos.....	10
Tema 1.2: Diseño y Arquitectura de los sistemas embebidos	14
1.2.1 Diseño basado en plataforma.....	14
El ciclo de vida del diseño de Sistemas Embebidos.....	14
Métricas de diseño.....	16
Restricciones de diseño.....	17
1.2.2 Arquitectura de Sistemas embebidos	20
Herramientas & Software de la asignatura.....	23
Bibliografía	24



Organización de la lectura para el estudiante por semana del compendio

Semanas	Paginas
Semana 1	Páginas 3 –13
Semana 2	Páginas 14 – 22
Semana 3	Repaso

Resultado de aprendizaje de la asignatura

Implementar un sistema embebido básico mediante el uso de herramientas de prototipado de hardware para el control de dispositivos que soporten soluciones en diferentes ámbitos.



Sistemas Embebidos



Unidad 1. FUNDAMENTOS DE SISTEMAS EMBEBIDOS

Resultado de aprendizaje de la unidad: Identificar las características prioritarias de los sistemas operativos para aplicaciones embebidas.

Tema 1.1: Conceptos Básicos de Sistemas Embebidos

Se lo puede definir ampliamente a un sistema embebido como un dispositivo que contiene componentes de hardware y software estrechamente acoplados para realizar una sola función. También se los llama “sistemas empotrados” (y en inglés *embedded systems*).

Estos sistemas normalmente forman parte de un sistema más grande, que realiza operaciones más completas. Por ejemplo, los vehículos modernos contienen muchos sistemas embebidos: uno controla el sistema antibloqueo de frenos, otro las emisiones, otro la información mostrada en el tablero, y así (Barr & Massa, 2013). Los sistemas embebidos no están diseñados para ser programado de forma independiente por el usuario, y se espera que funcione con un mínimo de o sin interacción humana (Perez, 2009). En la Figura 1 se muestran los elementos físicos de un posible sistema embebido.

1.1.1 Características esenciales de los sistemas embebidos.

Un sistema embebido tiene tres componentes principales:

- El hardware, generalmente simple y de tamaño reducido.

- Un software primario o aplicación principal. Este software o aplicación lleva a cabo una tarea en particular, o en algunas ocasiones una serie de tareas.
- Un sistema operativo, que permite supervisar la(s) aplicación(es), además de proveer los mecanismos para la ejecución de procesos. Respecto a este sistema operativo:
 - No tiene tantas opciones como el de un equipo de cómputo de escritorio.
 - En muchos sistemas embebidos es requerido que el sistema operativo posea características de tiempo real.
 - En otros sistemas embebidos más simples no es necesario un sistema operativo completo (Perez, 2009).

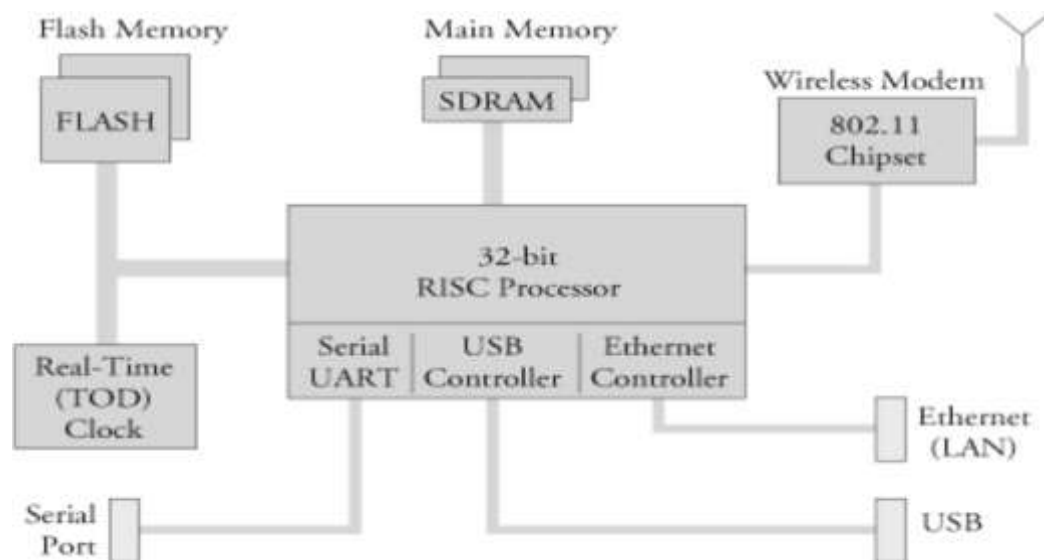


Figura 1. Esquema de un ejemplo de un sistema embebido (Perez, 2009).

Los sistemas embebidos poseen ciertas características que los distinguen de otros sistemas de cómputo. Entre otras, se destacan 3 características principales: Funcionamiento específico, fuertes limitaciones, reactivos y tiempo real.

Funcionamiento específico.

Un sistema embebido usualmente ejecuta un programa específico de forma repetitiva. Por ejemplo una alarma, siempre en una alarma. En contraste, un sistema de escritorio ejecuta una amplia variedad de programas, como hojas de cálculo, juegos, etc.; además nuevos programas son añadidos frecuentemente.

Por supuesto, puede haber excepciones. Podría ocurrir que el programa del sistema embebido fuese actualizado a una nueva versión. Por ejemplo, un teléfono celular podría actualizarse de alguna manera.

Fuertes limitaciones/restricciones.

Todos los sistemas de computación poseen limitaciones en sus métricas de diseño, pero en los sistemas embebidos son muy fuertes. Una métrica de diseño es una medida de algunas características de implementación, como: costo, tamaño, desempeño, y consumo de energía.

Los sistemas embebidos generalmente deben ser poco costosos, poseer un tamaño reducido, tener un buen desempeño para procesar datos en tiempo real, y además consumir un mínimo de energía para extender el tiempo de vida de las baterías o prevenir la necesidad de elementos adicionales de enfriamiento.

Reactivos y de tiempo real.

Muchos sistemas embebidos deben ser reactivos o reaccionar ante cambios en el ambiente, además de realizar algunos cálculos en tiempo real sin ningún retraso, es decir, se deben tener resultados en tiempos fijos ante cualquier eventualidad. Por ejemplo, el módulo de control de viaje de un automóvil continuamente monitorea la velocidad y los sensores de frenos, reaccionando ante cualquier eventualidad. Ante un estímulo anormal, el módulo de control debe realizar los cálculos de forma precisa y acelerada para garantizar la entrega de los resultados dentro de un tiempo límite, una violación en este tiempo podría ocasionar la pérdida del control del automóvil.

En contraste, un sistema de escritorio se enfoca en realizar cálculos con una frecuencia no determinada y la demora de los mismos no suelen producir fallas en el sistema (Perez, 2009).



Comprueba tu aprendizaje: Piensa en que aparatos o dispositivos, de uso cotidiano, son sistemas embebidos.



Video: En este video puedes revisar de nuevo que son los sistemas embebidos y sus características <https://youtu.be/-ajcVtBx6qY?t=40>.

1.1.2 Breve historia de los sistemas embebidos

Al inicio de la década de 1970 se empiezan a desarrollar los primeros microprocesadores. Empresas como Texas Instruments, Intel, y el US Navy desarrollaron sus microprocesadores, de manera independiente y casi simultáneamente.

Estos primeros microprocesadores empezaron a incorporar en el mismo chip una memoria ROM y una RAM, ofreciendo la funcionalidad de una computadora en un simple chip. Surge así el microcomputador-en-un-chip, o **microcontrolador**. Algunos de estos chips fueron el TMS1000 de Texas Instruments, o el 4004 de Intel (Figura 2), ambos microprocesadores de 4 bits.

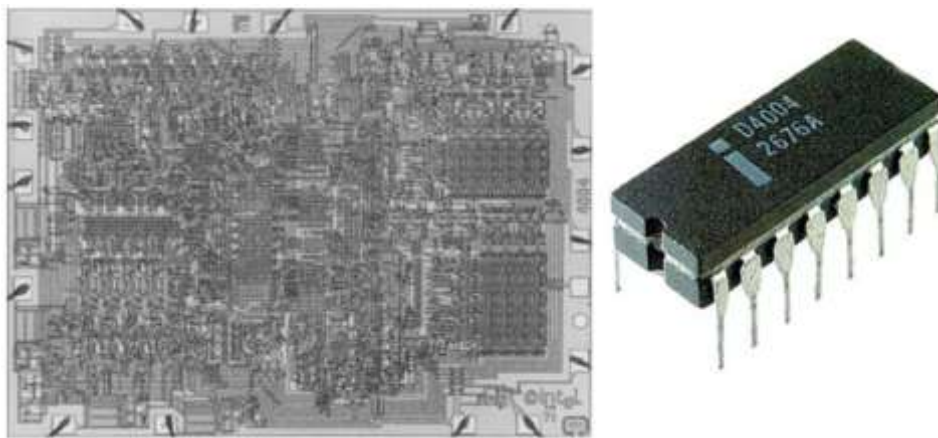


Figura 2. Fotografía de un chip Intel 4004.

A finales de los 70, los chips de 8 bits dominaban el mercado, entre otros el Intel 8080, Zilog Z-80, o el Motorola 6800. Su aplicación era principalmente en dispositivos de cálculo y de informática.

La evolución de los CPUs siguió en los 80s y 90s, apareciendo diseños de 16, 32 y 64 bits, e incluso diseños especializados de 128 bits. Ejemplos de fabricantes que en los últimos años se destacan, incluyen Texas Instruments, Intel, Microchip, Freescale/Motorola, Zilog, Advanced Micro Devices (AMD), MIPS Technologies, entre otros. Hoy en día existen procesadores de 8, 16, 32, y 64 bits. Su uso es muy amplio, no solo en informática, sino en electrodomésticos, maquinarias, dispositivos digitales, vehículos de todo tipo, entre otras aplicaciones (Jiménez, Palomera, & Couvertier, 2014).

A pesar de la diversidad de fabricantes y procesadores, y del desarrollo de estos últimos, las aplicaciones de sistemas embebidos han sido dominadas por décadas por

procesadores de 8 y 16 bits. La Figura 3 muestra una representación de las ventas (en cantidad y volumen de costo) de los CPUs en la primera parte de la década de 2010. A pesar de que el mayor volumen de venta correspondía a procesadores de 32 bits; en número de unidades los procesadores de 4, 8 y 16 bits constituyen más de las tres cuartas partes del total. Esto debido a que se trata de CPUs incluidos en microcontroladores de uso masivo, principalmente en electrodomésticos, juguetes, vehículos, entre otros aparatos de uso cotidiano.



Figura 3. Distribución del mercado de procesadores a inicios de la década de 2010 (Jiménez, Palomera, & Couvertier, 2014).

En años recientes, especialmente con el auge del Internet de las Cosas o IoT (*Internet of Things*) el uso de CPUs de mayor número de bits se ha vuelto común en el área de la informática. En encuestas realizadas a profesionales del área, se aprecia que el principal tipo de procesador utilizado en nuevos proyectos es de 32 bits (Figura 4), aunque no implica que no se utilicen CPUs de otros tipos (Aspencore, 2019).

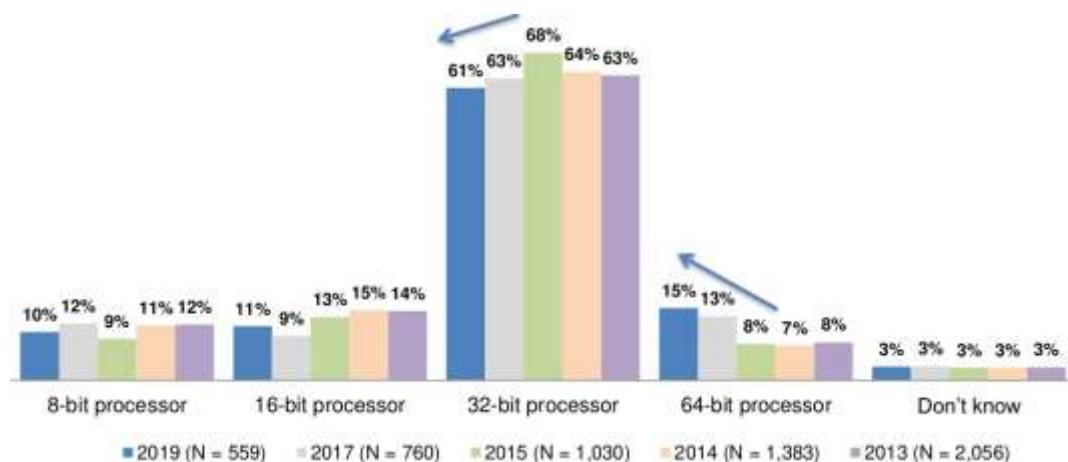


Figura 4. Principal CPU utilizados en proyectos embebidos (Aspencore, 2019).



Sabías que: Además de los fabricantes nombrados, existen muchos otros más, algunos enfocados en chips para aplicaciones específicas, como por ejemplo Qualcomm, que fabrica microcontroladores para dispositivos móviles, o NVidia, que se enfoca en chips de procesamiento gráfico.



Observaciones: No es que haya contradicción entre los datos de la Figura 3 y los de la Figura 4. Si bien la mayoría de los actuales proyectos embebidos consideran el uso de CPUs de 32 o más bits, el número total de dispositivos que utilizan estos procesadores siguen siendo menor a los que utilizan CPUs de 8 o 16 bits.

1.1.3 Tipos de Sistemas Embebidos

En general hay 3 categorías de sistemas embebidos: Pequeños, Distribuidos, y de Alto rendimiento (*High-Performance*) (Figura 5).

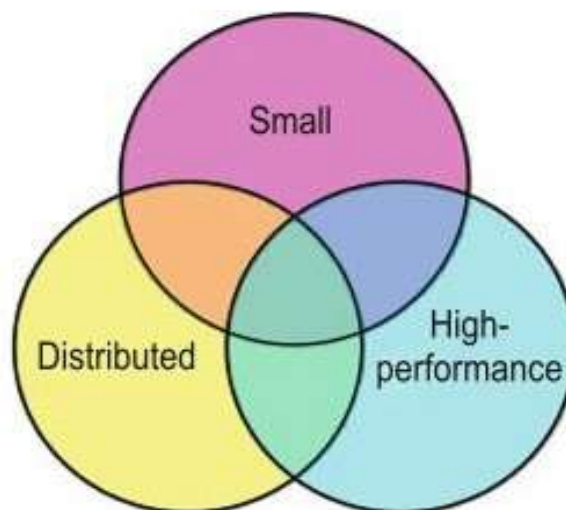


Figura 5. Clasificación de Sistemas Embebidos (Barr & Massa, 2013).

Sistemas Embebidos Pequeños.

Es el tipo más común de sistema embebido. Generalmente se centra en un solo chip de microcontrolador que controla toda la aplicación. El TMS1000 de Texas Instruments puede ser considerado el primero de este tipo.

Están altamente integrados, agregando solo algunos componentes analógicos, sensores, actuadores e interfaz de usuario, según sea necesario. Operan con un mantenimiento mínimo o nulo, son de muy bajo costo y se producen en grandes cantidades.

Suele ser de una sola tarea y rara vez requiere un RTOS (*Real Time Operating System*).

Ejemplos de estos sistemas: monitoreo de presión de llantas, controladores de hornos microondas o tostadoras, controladores de juguetes electrónicos, entre otros.

Sistemas Embebidos Distribuidos.

No confundir con sistema informático distribuido, pues este tipo de sistema es distinto. En estos sistemas la CPU reside en un chip separado mientras que el resto de componentes (como memorias, E/S, los coprocesadores y otras funciones especiales) se distribuyen en uno o varios chips, lo cual es denominado el chipset del procesador.

Típicamente estos sistemas requieren mantenimiento y actualizaciones, e incluyen algunos medios de diagnóstico de sistemas, aunque la robustez no es un tema crítico. Por lo general, administran múltiples tareas, por lo que el uso de RTOS para el desarrollo de sistemas es común. Los volúmenes de producción son relativamente altos, mientras que el costo de los mismos está relacionado al desempeño esperado.

Ejemplos de estos sistemas: procesadores de video, controladores de videojuegos, registradores de datos y procesadores de red, entre otros.

Sistemas Embebidos de Alto Rendimiento.

Son sistemas embebidos altamente especializados, para aplicaciones que requieren cálculos rápidos, robustez, tolerancia a fallas y alta capacidad de mantenimiento.

Generalmente requieren un ASICs (*Application Specific Integrated Circuit*) dedicados, y son típicamente distribuidos, y pueden incluir otros componentes como DSPs (*Digital Signal Processor*) o FPGAs (*Field Programmable Gate Array*). En muchos casos se hace obligatorio el uso de RTOS para gestionar la multiplicidad de tareas. Se producen en pequeñas cantidades y su coste es muy elevado.

Se utilizan en aplicaciones militares y aeroespaciales, como controladores de vuelo, sistemas de guía de misiles y sistemas de navegación de naves espaciales (Jiménez, Palomera, & Couvertier, 2014).



Sabías que: Entre los tipos de sistemas que hemos visto podemos encontrar "zonas grises", como vemos en la Figura 5, donde un sistema embebido puede ser categorizado como más de un tipo. Además, puede haber otros criterios de clasificación.

1.1.4 Aplicación de Sistemas Embebidos.

Los sistemas embebidos se encuentran en una variedad de dispositivos electrónicos comunes, tales como consumibles electrónicos (teléfonos celulares, pagers, cámaras digitales, video juegos portátiles, calculadoras, etc.), electrodomésticos (hornos microondas, máquinas contestadoras, termostatos, lavadoras, etc.), equipos de oficina (fax, copiadoras, impresoras, scanners), equipos de negocios (caja registradora, sistemas de alarma, lectores de tarjeta y cajeros automáticos), y automóviles (control de transmisión, control de viaje, inyección de combustible, ABS, etc.). Podría decirse que prácticamente cualquier dispositivo que se ejecute con electricidad o ya tiene un sistema computacional embebido o próximamente lo tendrá (Perez, 2009).

La aplicación de sistemas embebidos es muy amplia, en la Figura 6 podemos ver la diversidad de áreas para las que se desarrolla estos dispositivos, siendo la automatización y control y la electrónica de consumo las áreas con mayor demanda. Sin embargo, también se utilizan sistemas embebidos en áreas tan diversas como la medicina, redes y comunicaciones, a nivel militar, transporte, entre otras.

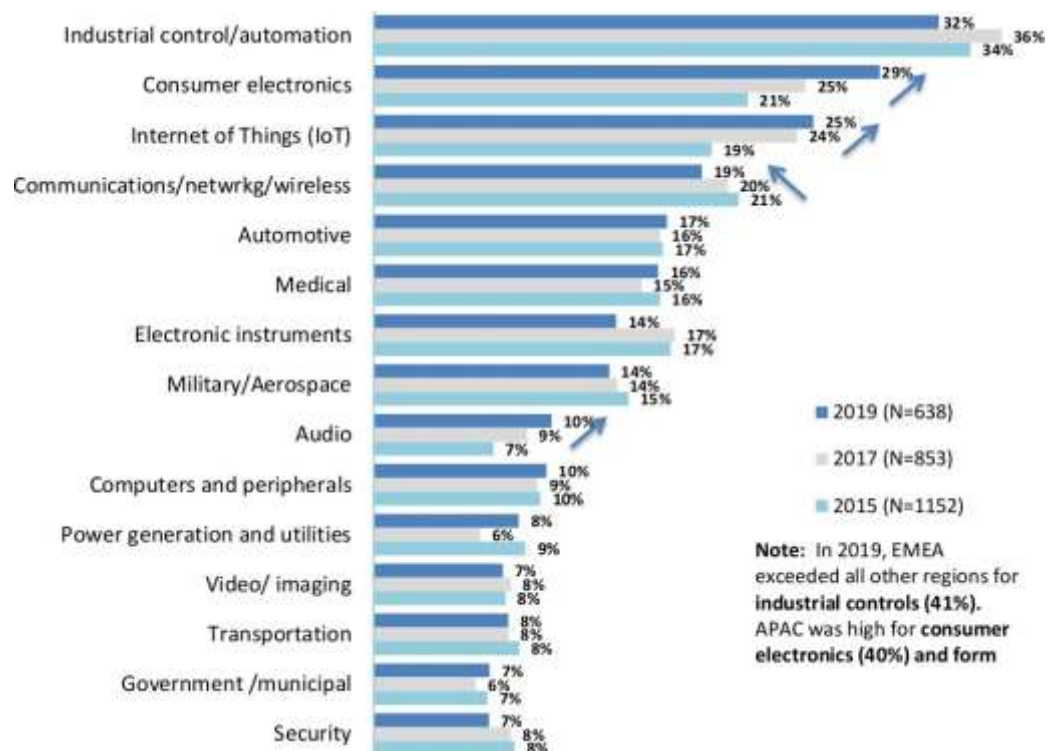


Figura 6. Áreas de aplicación de proyectos de sistemas embebidos (Aspencore, 2019).

Por otro lado, aunque los lenguajes de programación que se emplean en el desarrollo de sistemas embebidos también pueden ser varios, los lenguajes que más se utilizan

suelen ser C y C++ (Figura 7). Como se verá más adelante, independientemente del lenguaje utilizado, es necesario tener un compilador adecuado para cada tipo o arquitectura de microcontrolador.

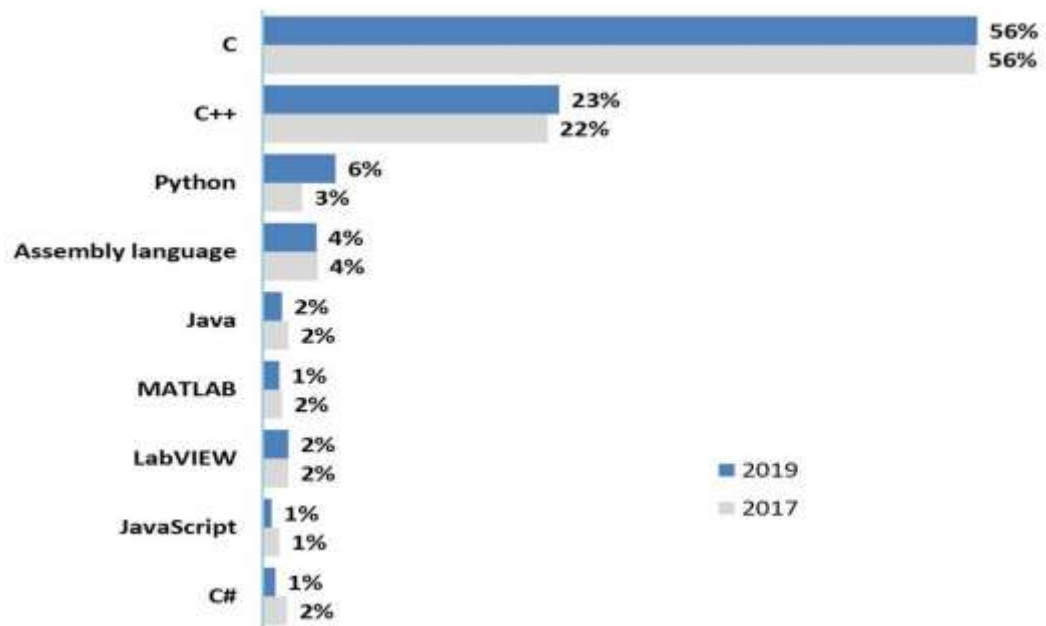


Figura 7. Lenguajes de programación utilizados en sistemas embebidos (Aspencore, 2019).

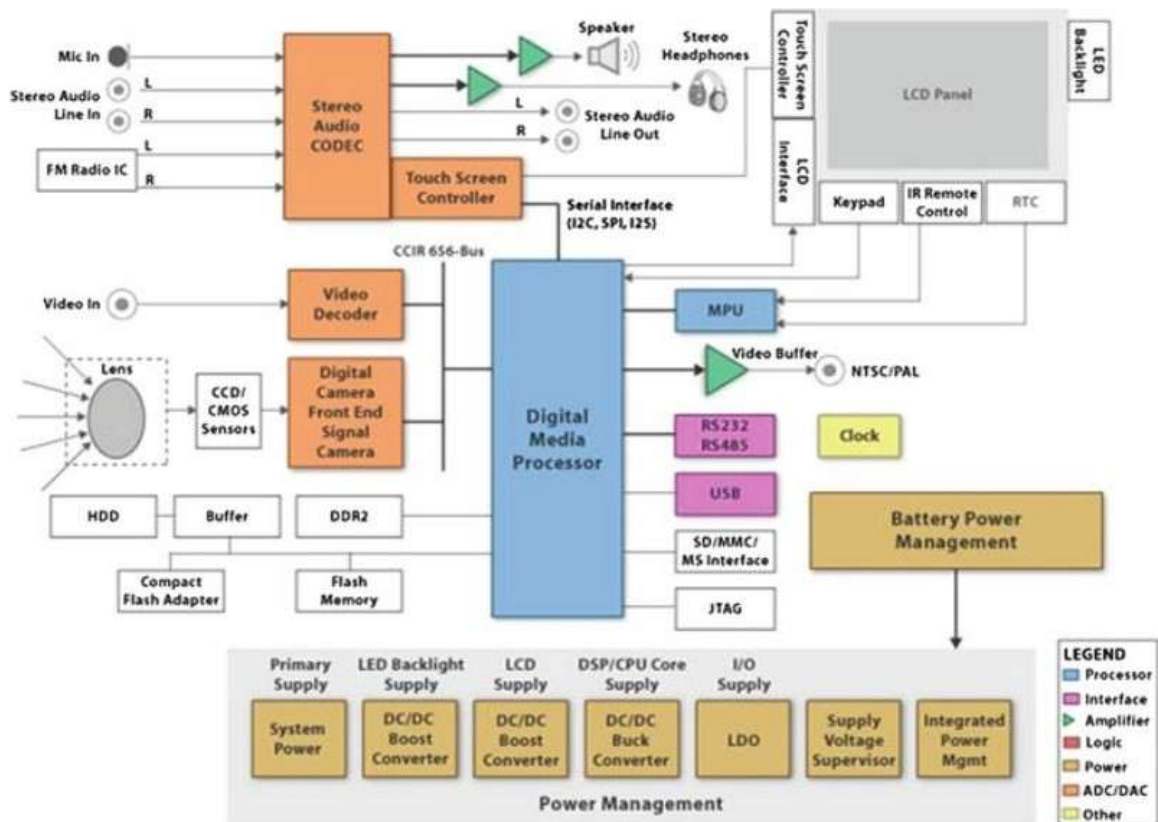


Figura 8. Esquema genérico de un reproductor multimedia.

La Figura 8 muestra el esquema de un sistema embebido para una aplicación específica (en este caso un reproductor multimedia). Así como los sistemas embebidos pueden encontrarse en casi cualquier dispositivo de uso diario, también es posible utilizar los mismos componentes de un sistema en otro dispositivo. Por ejemplo, en el caso del reproductor multimedia de la Figura 8, la mayoría de sus elementos podrían utilizarse para un portero eléctrico con video, o para un monitor de signos vitales.

Dentro del esquema de un dispositivo embebido (como el de la Figura 8), se pueden identificar tres tipos de componentes:

- Sensores: vienen a ser la parte de **entrada** de datos del sistema embebido. Toman o miden datos del entorno del sistema, que son pasados a la parte de procesamiento.
- Procesamiento: es la parte del sistema embebido (usualmente un microcontrolador) que toma los datos de los sensores, los procesa, y realiza alguna acción.
- Actuadores: son la parte de **salida** del sistema. Son utilizados por el microcontrolador para realizar alguna acción (activar un motor, emitir un sonido, mostrar un mensaje en pantalla, entre otros.).

Además, muchos sistemas embebidos tienen un componente de comunicación para intercambiar información con otros dispositivos, pero eso lo veremos más adelante.

Una herramienta para diseñar circuitos de sistemas embebidos es la aplicación en línea Circuito.io (<https://www.circuito.io/>) (Figura 9). Esta aplicación, desarrollada por la empresa Roboplan, brinda la oportunidad al usuario de concentrarse en la esencia de una idea de diseño de sistema embebido, mientras la aplicación resuelve los detalles esenciales del diseño y calcula automáticamente una lista detallada de materiales, instrucciones de cableado y código de muestra para que se pueda comenzar a construir.



Figura 9. Sitio web de circuito.io.

Con esta herramienta no son necesarias tantas búsquedas de muestras de código y esquemas que no funcionarán. La aplicación es gratis, y permite tener las herramientas iniciales para construir el circuito para un sistema embebido (Circuito.io, 2021).

 **Video:** En este video se revisa de nuevo en qué consisten los sistemas embebidos, pero además se indican otros aspectos sobre sus características y tendencias actuales <https://www.youtube.com/watch?v=8n1NdLWk8GY>.



Tema 1.2: Diseño y Arquitectura de los sistemas embebidos

1.2.1 Diseño basado en plataforma

El ciclo de vida del diseño de Sistemas Embebidos.

Los sistemas embebidos tienen una vida útil finita a lo largo de la cual pasan por diferentes etapas, desde la concepción o el nacimiento del sistema hasta la eliminación y, en muchos casos, el renacimiento. A esto lo llamamos el ciclo de vida de un sistema integrado. Se pueden identificar cinco etapas en el ciclo: Concepción o Nacimiento, Diseño, Crecimiento, Madurez y Decadencia. La Figura 10 ilustra la secuencia de estas etapas y las fases críticas que componen cada etapa.

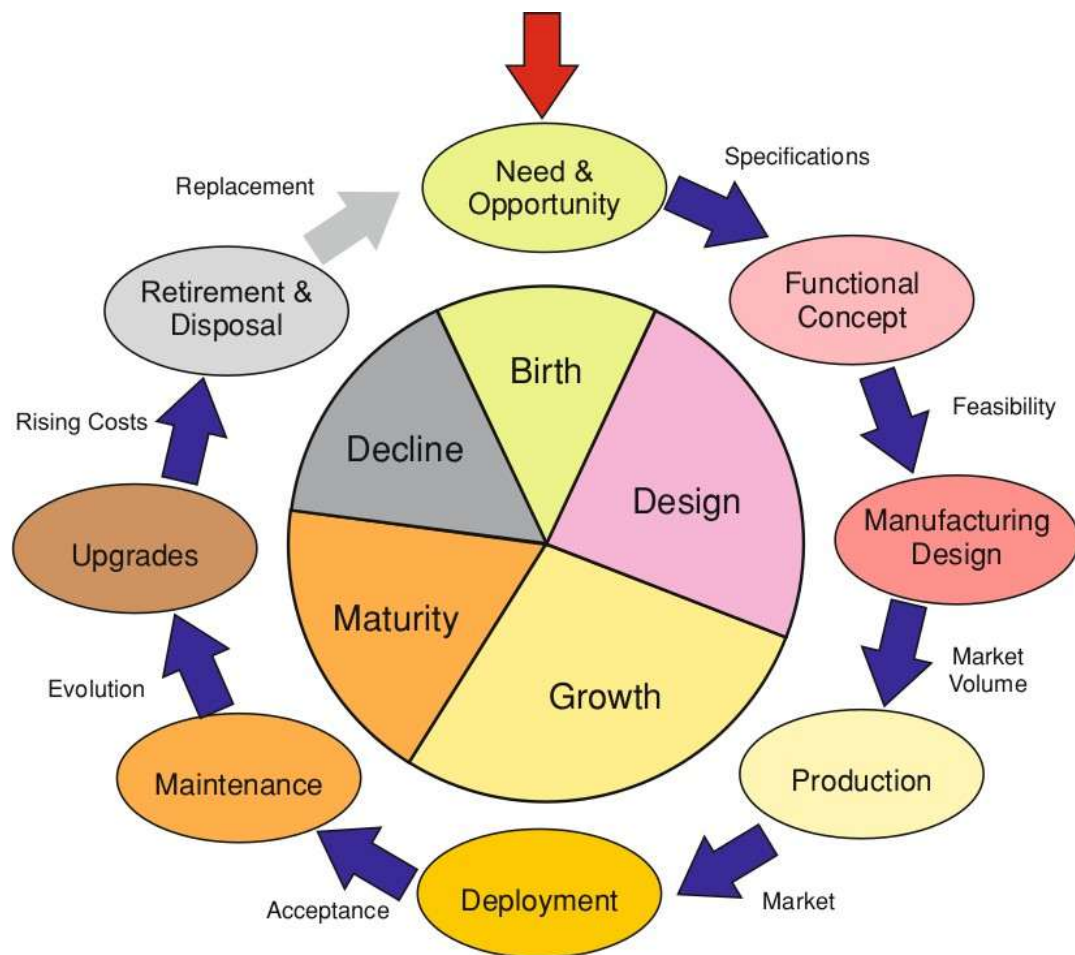


Figura 10. Ciclo de vida de un sistema embebido (Jiménez, Palomera, & Couvertier, 2014).

Un diseño embebido se concibe primero por la identificación de la necesidad de resolver un problema particular, o la oportunidad de aplicar la tecnología embebida a un problema antiguo. Muchos sistemas embebidos se conciben por la necesidad de

cambiar la forma en que se resuelven viejos problemas o la necesidad de brindarles soluciones rentables. Otros sistemas se conciben con los propios problemas. La oportunidad surge con la realización de soluciones eficientes habilitadas por el uso de sistemas integrados. En cualquier caso, la aplicación en la que se debe integrar el diseño dicta las especificaciones que conducen a la etapa de diseño.

En la etapa de diseño, el sistema pasa primero por una fase de **diseño funcional**, donde se desarrollan prototipos de prueba de concepto. Esta fase permite ajustar la funcionalidad del sistema para que se ajuste a la aplicación para la que fue diseñado. Un análisis de factibilidad que considere variables como el costo del producto, la ventana de mercado y la obsolescencia de los componentes determina si procede o no la fabricación y el diseño del producto. Esta fase establece la forma en que un prototipo se convierte en producto y la logística para su fabricación. El diseño es, con mucho, la etapa más costosa en el ciclo de vida de un sistema embebido, ya que aquí surgen la mayoría de los costos de ingeniería no recurrentes.

La etapa de crecimiento se inicia con la producción del sistema para abastecer una demanda estimada del mercado. La fase de implementación implica la distribución, instalación y configuración del sistema. La adecuación del sistema a sus necesidades originarias define su aceptación, lo que lleva al sistema integrado a su etapa de madurez.

En la etapa de madurez, el producto se mantiene funcional y actualizado. Esto implica proporcionar soporte técnico, mantenimiento periódico y servicio. Se espera que la etapa de madurez permita ejecutar el producto con costos mínimos. A medida que el sistema comienza a envejecer, los costos de mantenimiento comienzan a aumentar, lo que indica su transición a la etapa de declive.

En la etapa de declive, los componentes del sistema se vuelven obsoletos y difíciles de obtener, lo que eleva el costo de mantenimiento, servicio y actualizaciones a niveles que se acercan o superan los de reemplazar todo el sistema a la vez. Este es el punto en el que el sistema debe retirarse del servicio y desecharse.

Los diseños de productos deben prever esta etapa para idear formas en las que el reciclaje y la reutilización puedan ser factibles, reduciendo los costos y el impacto de su eliminación. En muchos casos, el reemplazo del sistema crea nuevas necesidades y

oportunidades para ser satisfechas por un nuevo diseño, reiniciando el ciclo de vida de un nuevo sistema (Jiménez, Palomera, & Couvertier, 2014).

Métricas de diseño.

El diseñador de un sistema embebido debe construir una implementación que satisfaga la necesidad específica. Una dificultad adicional es construir una implementación que a la vez optimice diversas métricas de diseño. Una implementación consiste en un microprocesador con un programa que lo acompañe, una conexión de compuertas digitales, o una combinación de ambos. Una métrica de diseño es una característica medible de la implementación del sistema. Métricas comunes incluyen:

- Costo NRE (*Nonrecurring Enginnering*). El costo monetario de diseñar el sistema por primera vez. Una vez que el sistema está diseñado, cualquier número de unidades puede ser manufacturado sin incurrir en costo adicionales de diseño.
- Costo unitario. El costo monetario de manufacturar cada copia del sistema, excluyendo el costo NRE.
- Tamaño. El espacio físico requerido para el sistema, usualmente medido en bytes para software, y compuertas o transistores para hardware.
- Desempeño. El tiempo de ejecución del sistema.
- Energía. La cantidad de energía consumida por el sistema, que puede determinar el tiempo de vida de la batería. Flexibilidad. La habilidad de cambiar la funcionalidad del sistema sin incurrir en un gran costo NRE.
- Tiempo para crear un prototipo. El tiempo necesario para construir una versión funcional del sistema.
- Tiempo de mercadeo. El tiempo requerido para desarrollar un sistema al punto en que pueda ser lanzado al mercado.
- Mantenimiento. La habilidad de modificar el sistema luego de su lanzamiento inicial, especialmente por diseñadores que no participaron en el diseño original.
- Correctitud. Confianza en que la funcionalidad del sistema es la correcta.
- Seguridad. La probabilidad de que el sistema no cause daño.

Las métricas típicamente compiten entre sí. La mejora de una métrica generalmente conlleva al degrado de otra. Por ejemplo, si se reduce el tamaño de una implementación su desempeño puede decaer como se ve en la Figura 11 (Perez, 2009).

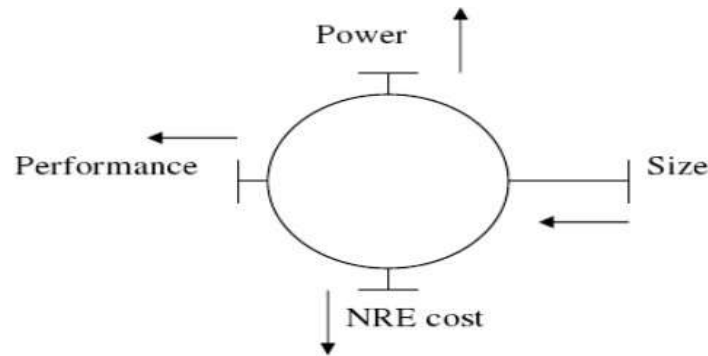


Figura 11. Competencia de métricas de diseño.

Restricciones de diseño.

Además de las métricas, el diseño debe considerar las restricciones típicas de los sistemas embebidos. Una gran mayoría de aplicaciones de sistemas embebidos terminan en el corazón de las aplicaciones electrónicas producidas en masa. Son sistemas con una alta sensibilidad de costo debido a los altos volúmenes en los que se producen. Los diseños deben completarse, fabricarse y lanzarse a tiempo para llegar a una ventana de mercado y maximizar los ingresos del producto. Por lo tanto, la lista de limitaciones que enfrentan los diseñadores proviene de diferentes perspectivas.

Las limitaciones más destacadas incluyen:

- **Funcionalidad:** La capacidad del sistema para realizar la función para la que fue diseñado. Verificar la funcionalidad requiere aproximadamente el 70% del tiempo de desarrollo. Se pueden aplicar diferentes métodos
 - Simulación de comportamiento
 - Simulación a nivel lógico y de circuito
 - Simulación de procesador
 - Depuradores JTAG o similares
 - Emulación en circuitos (FPGA)
 - Otras herramientas
- **Costo:** Recursos necesarios para concebir, diseñar, producir, mantener y descartar un sistema integrado.
 - En cierto sentido, el costo total C_T de producir V volumen de unidades de un sistema embebido puede calcularse como $C_T = NRE + (RP \cdot V)$, donde RP representa los costos de Producción Variable, y NRE es el costo de Ingeniería no recurrente.

- Desarrollar un sistema embebido desde cero es un proceso costoso, por lo que muchos proyectos se realizan utilizando componentes disponible comercialmente. Se puede utilizar el Modelo de diseño de flujo basado en partes COTS (*commercial, off-the-shelf*) (Figura 12). Aunque hay otros modelos, este suele ser muy utilizado, ya que se minimizan los costos de desarrollar hardware a través del uso de componentes ya existentes.

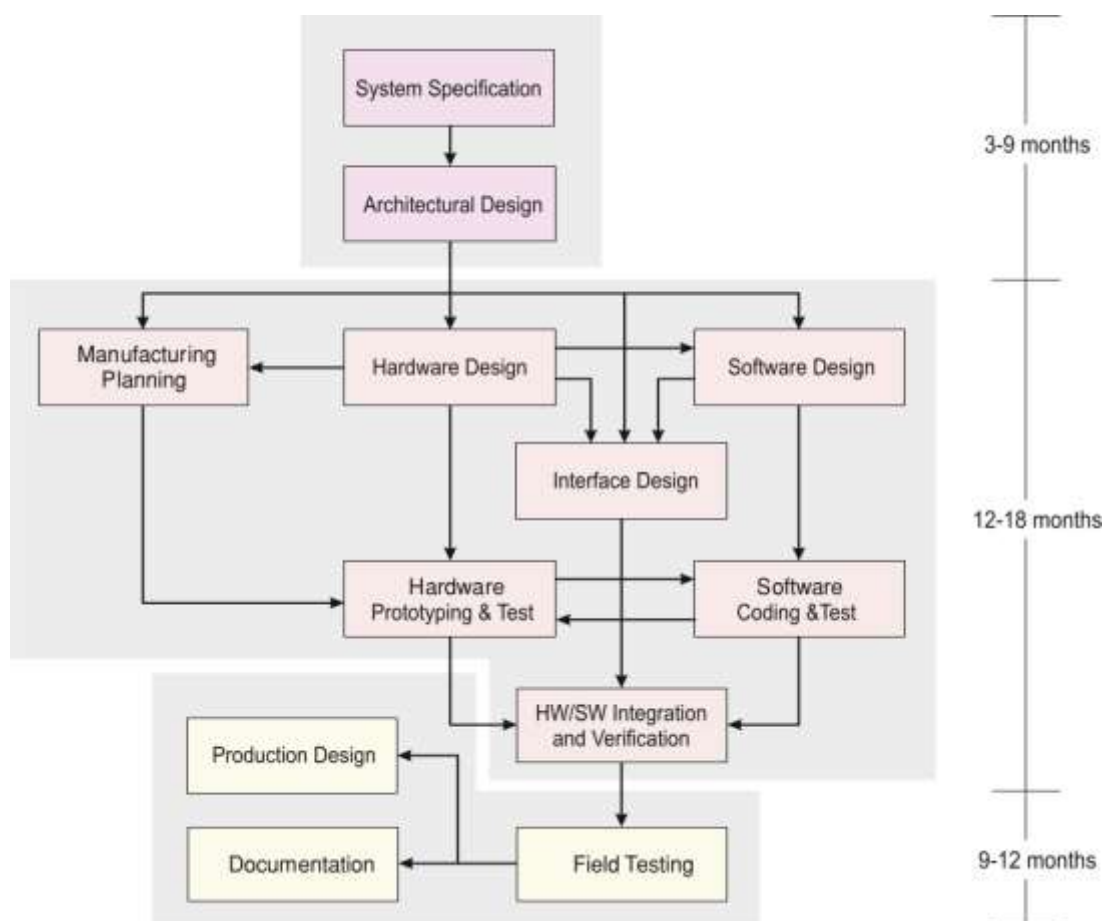


Figura 12. Modelo COTS para diseño de sistemas embebidos (Jiménez, Palomera, & Couvertier, 2014).

- **Rendimiento:** La capacidad del sistema para realizar su función a tiempo. Una forma de verlo es midiendo el número de operaciones por unidad de tiempo. Involucra tanto aspectos de hardware como de software:
 - Factores de Hardware: Velocidad de reloj, Arquitectura (CISC o RISC), Retardo de componentes, Modos de bajo consumo (Low-Power)
 - Factores de Software: Complejidad de algoritmos, Planificación de tareas múltiples, Comunicación entre tareas, Nivel de paralelismo implementado
- **Tamaño:** Espacio físico que ocupa una solución de sistema. En ciertas aplicaciones este es un factor muy importante.

- **Potencia y energía:** Necesaria para realizar su función. La energía en los sistemas embebidos se ha convertido en una restricción crítica, no solo en los sistemas portátiles que funcionan con baterías, sino también para todos los diseños de sistemas. La disipación de energía promedio de un diseño integrado define la tasa a la que el sistema consume energía. En aplicaciones alimentadas por batería, esto determina cuánto tiempo lleva agotar la capacidad de sus baterías. Pero además de la duración de la batería, la energía afecta muchos otros problemas en el diseño de sistemas integrados. Algunos de los temas más representativos incluyen:
 - Confiabilidad del sistema
 - Requisitos de refrigeración
 - Diseño de fuente de alimentación
 - Tamaño, peso y forma del sistema
 - Impacto medioambiental
-
- **Time to Market (TTM):** El tiempo desde la concepción del sistema hasta la implementación. El TTM se convierte en una restricción crítica cuando se tiene poco tiempo para sacar al mercado un producto.
- **Mantenibilidad:** capacidad del sistema para mantenerse funcional durante el mayor tiempo de su vida útil. La mantenibilidad en los sistemas embebidos se puede definir como una propiedad que permite actuar sobre el sistema, para garantizar un funcionamiento fiable hasta el final de su vida útil. Esta propiedad puede considerarse como una restricción de diseño porque, para que se habilite la mantenibilidad, debe planificarse desde la propia concepción del sistema.
 - La restricción de mantenibilidad puede tener diferentes niveles de relevancia según el tipo de sistema integrado que se considere. Los sistemas pequeños, económicos y de corta duración, como los teléfonos celulares y los pequeños electrodomésticos, generalmente se reemplazan antes de que necesiten mantenimiento. En este tipo de aplicaciones, una operación confiable depende esencialmente del nivel de verificación que se les haga antes de su despliegue. Los sistemas integrados grandes, costosos y complejos, como los que se usan en aviones o equipos médicos grandes, son críticos para la seguridad y se espera que

permanezcan en funcionamiento durante décadas. Para esta clase de sistemas, una operación confiable depende de la capacidad del sistema para mantenerse. Desde una perspectiva amplia, la mantenibilidad debe proporcionar al sistema los medios para ejecutar cuatro tipos básicos de acciones:

- Acciones correctivas.
- Acciones Adaptativas.
- Acciones Perfectivas.
- Acciones preventivas (Jiménez, Palomera, & Couvertier, 2014).



Sabías que: Aspectos como el consumo energético o el TTM pueden estimarse mediante ecuaciones.

1.2.2 Arquitectura de Sistemas embebidos

Independientemente de su función, un sistema embebido tiene como estructura dos conjuntos principales de componentes estrechamente acoplados:

- Un conjunto de componentes de hardware, que incluyen una unidad central de procesamiento, típicamente en forma de microcontrolador.
- Y una serie de programas de software, normalmente incluidos como firmware, que dan funcionalidad al hardware.

En la Figura 13 se describen estos dos componentes principales y su interrelación.

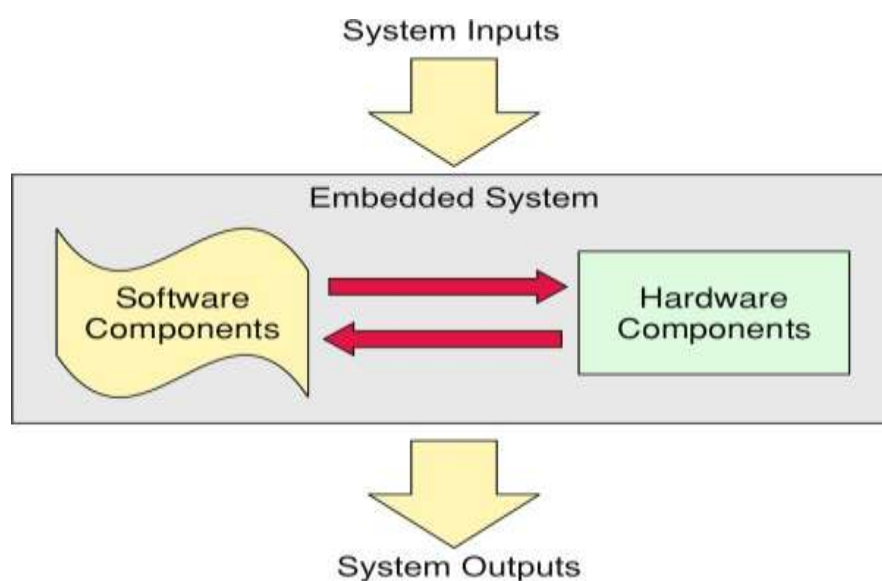


Figura 13. Interrelación entre componentes de un sistema embebido (Jiménez, Palomera, & Couvertier, 2014).

Sobre ambos conjunto de componentes se han establecido arquitecturas que definen su funcionamiento. En un sistema embebido, a diferencia de lo que sucede con los sistemas de cómputo tradicionales en la actualidad, se suele poner mayor énfasis en la estructura del hardware. En la Figura 14 se muestra un ejemplo de los distintos componentes de hardware que componen un sistema embebido.

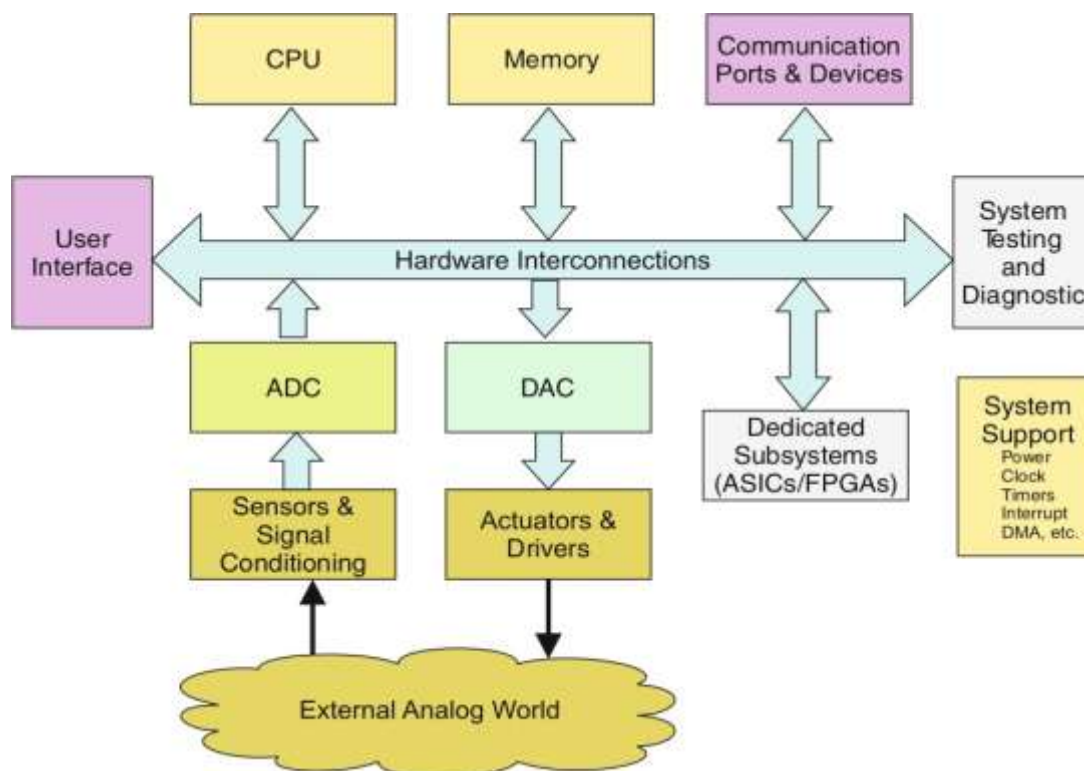


Figura 14. Ejemplo de componentes de hardware en un sistema embebido (Jiménez, Palomera, & Couvertier, 2014).

Más adelante, en las siguientes unidades, se hablará más de las arquitecturas relacionadas a los sistemas embebidos, especialmente a nivel de los microcontroladores sobre los que estos se construyen. Por ahora veremos un ejemplo de la arquitectura de sistemas de cómputo, aplicables también a sistemas embebidos.

Un criterio de diseño de sistemas embebidos es cuál de los dos componentes, hardware o software, debe optimizarse. Bajo este punto de vista, se puede hablar de las arquitecturas CISC y RISC:

- Las máquinas CISC (*Complex Instruction Set Computing*) se caracterizan por palabras de instrucción de longitud variable, es decir, con diferente número de bits, tamaños de código pequeños y múltiples instrucciones complejas sincronizadas a nivel de máquina. La arquitectura CISC se enfoca en lograr tanto como sea posible con cada instrucción, con el fin de generar programas simples.

Este enfoque ayuda a la tarea del programador al tiempo que aumenta la complejidad del hardware.

- Las máquinas RISC (*Reduced Instruction Set Computing*), por otro lado, están diseñadas con un enfoque en instrucciones simples, incluso si eso da como resultado programas más largos. Esta orientación simplifica la estructura del hardware. El diseño espera que la ejecución de una sola instrucción se reduzca, como máximo, un solo ciclo de memoria de datos, en comparación con las “instrucciones complejas” de un sistema CISC (Jiménez, Palomera, & Couvertier, 2014).



Recuerda que: La Arquitectura de un componente o sistema se refiere a como está estructurado dicho sistema o componente.



Video: En este video puedes revisar una explicación sobre la diferencia entre CISC y RISC <https://www.youtube.com/watch?v=3y3QFIxOJYU>.



Herramientas & Software de la asignatura

- **Arduino IDE**, entorno de programación para sistemas embebidos. Disponible para descargar en <https://www.arduino.cc/en/software>.
- **Circuito.io**, entorno en línea para diseñar circuitos con componentes. Disponible en <https://www.circuito.io/>.
- **Proteus Circuit Simulation** (opcional), software para diseñar y simular circuitos embebidos. Es de pago, pero hay una versión gratuita limitada, disponible en <https://www.labcenter.com/downloads/>.

Bibliografía

- Aspencore. (2019). *2019 Embedded Market Study: Integrating IoT and Advanced Technology Designs, Application Development PRocessing Enviroments*.
- Barr, M., & Massa, A. (2013). *Programming Embedded Systems*. O'Reilly.
- Circuito.io. (2021). *We are circuito.io*. Recuperado el 12 de 05 de 2022, de <https://www.circuito.io/about>
- Jiménez, M., Palomera, R., & Couvertier, I. (2014). *Introduction to Embedded Systems*. Springer.
- Perez, D. (2009). *Sistemas Embebidos y Sistemas Operativos Embebidos. Lecturas en Ciencias de la Computación*.

Este compendio recoge textualmente documentos e información de varias fuentes debidamente citadas, así como referencias elaboradas por el autor para conectar los diferentes temas.

Se lo utiliza únicamente con fines educativos.